SE LAB 04 — VIBE CODING

Fruit Ninja Python/Pygame Code-Change Chat History

GitHub repository: https://github.com/SETAPESU26/41_fruit-ninja

NAME: ABHISHEK
SRN: PES1UG24CS650

Project Overview

The project is a Fruit Ninja–style game built with Python and Pygame. The four tasks improve input collision handling, end-of-round feedback, replay/difficulty selection, and audio feedback.

- Task 1: Improve fast-swipe collision detection.
- Task 2: Display a Game Over screen with the final score and instructions.
- Task 3: Add Easy, Medium, and Hard difficulty levels and replay/reset behavior.
- Task 4: Generate and play sound effects in memory.

Task 1 — Fast-Swipe Collision Detection

File(s): game/game_engine.py (related helper: game/fruit.py)

1. User — Task Request

The game sometimes failed to slice a fruit when the mouse moved very quickly. Improve collision detection so a fast swipe is less likely to pass over a fruit between consecutive mouse-motion events.

2. Assistant — Explanation

The previous and current mouse positions define a line segment for the swipe. The updated _handle_motion() checks intermediate points along that segment against each unsliced fruit. If a point falls inside the fruit's circular hitbox, the existing _slice() method is called. The current position is then added to the swipe trail, which is kept to a maximum of 15 points.

3. Assistant — Core Implementation / Code Changes

Code excerpts are source-based and shortened for readability; they are not presented as a complete source file.

```
# Source-based excerpt: _handle_motion() in game/game_engine.py
# Excerpt shown for the relevant collision-detection logic.

def _handle_motion(self, pos):
    previous_pos = self.trail[-1] if self.trail else pos

    x1, y1 = previous_pos
    x2, y2 = pos

    for fruit in self.fruits:
        if fruit.sliced:
            continue

        # Check several points along the swipe segment
```

```
        # so fast mouse movements cannot skip over a fruit.
        steps = max(abs(x2 - x1), abs(y2 - y1), 1)

        for i in range(steps + 1):
            t = i / steps

            x = x1 + (x2 - x1) * t
            y = y1 + (y2 - y1) * t

            if fruit.contains_point(x, y):
                self._slice(fruit)
                break

    self.trail.append(pos)

    if len(self.trail) > 15:
        self.trail.pop(0)


# Related hit-test helper in game/fruit.py
def contains_point(self, x, y):
    return math.hypot(self.x - x, self.y - y) <= self.radius
```

4. Assistant — Why It Works

Sampling the swipe segment checks more than just the mouse event's endpoint. This reduces missed collisions during fast movement. The contains_point() helper uses the distance from the point to the fruit's centre and compares it with the fruit radius. Already-sliced fruits are skipped, and reusing _slice() preserves the game's existing fruit/bomb behavior.

5. Testing Checklist

- ☐ Run the game with `python main.py`.
- ☐ Try slow swipes across fruit.
- ☐ Try fast swipes across fruit and check that hits are registered more reliably.
- ☐ Confirm the score increases when a fruit is sliced.
- ☐ Check that bomb behavior still works.

6. Task Outcome and Git Commit Message

Outcome: Swipe collision checks sample intermediate positions to reduce missed fruit hits during fast mouse movement.

Recorded commit message: `Fix fast-swipe collision detection`

Task 2 — Game Over Screen

File(s): game/game_engine.py

1. User — Task Request

Add a visible Game Over screen when a round ends. It should show the final score and explain the available difficulty/replay controls.

2. Assistant — Explanation

The render() method checks self.game_over. When the flag is true, it draws a translucent dark overlay, displays a large GAME OVER title, and presents the final score, difficulty-selection instructions, and replay instructions. The excerpt below shows the overlay and title portion; the final-score and instruction-rendering lines are also part of the implementation but are not reproduced in full here.

3. Assistant — Core Implementation / Code Changes

Code excerpts are source-based and shortened for readability; they are not presented as a complete source file.

```
# Source-based excerpt: relevant part of render() in game/game_engine.py
# This excerpt is not the complete render() method.

if self.game_over:
    overlay = pygame.Surface(
        (self.width, self.height),
        pygame.SRCALPHA
    )

    overlay.fill((0, 0, 0, 180))
    screen.blit(overlay, (0, 0))

    game_over_font = pygame.font.SysFont(
        "Arial", 64, bold=True
    )

    game_over_text = game_over_font.render(
        "GAME OVER", True, WHITE
    )

    game_over_rect = game_over_text.get_rect(
        center=(
            self.width // 2,
            self.height // 2 - 110
        )
    )

    screen.blit(game_over_text, game_over_rect)

# The implementation also renders:
# f"Final Score: {self.score}"
# difficulty-selection instructions
# replay instructions
```

4. Assistant — Why It Works

The translucent overlay darkens the game view without replacing it with a completely separate window. The title is centred horizontally and positioned above the vertical centre. Showing the final score and controls tells the player why the round ended and what they can do next.

5. Testing Checklist

- ☐ Slice a bomb and confirm that the Game Over screen appears.
- ☐ In a separate round, lose all lives and confirm that the Game Over screen appears.

- ☐ Verify that the final score is displayed.
- ☐ Check that the difficulty-selection and replay instructions are visible.

6. Task Outcome and Git Commit Message

Outcome: The render method displays an end-of-round overlay, final score, and player instructions.

Recorded commit message: `Add Game Over screen`

Task 3 — Replay and Difficulty Levels

File(s): game/game_engine.py (window-close handling is in main.py)

1. User — Task Request

Allow the player to start another round after Game Over and choose Easy, Medium, or Hard. Reset the old round's state when a new difficulty is selected.

2. Assistant — Explanation

set_difficulty() assigns the spawn interval, speed scale, and bomb probability for the selected level. reset_game() clears the fruit list and swipe trail, resets the spawn timer, restores three lives and a score of zero, clears the Game Over flag, and applies the selected difficulty. While Game Over is active, E selects Easy, M selects Medium, and H selects Hard. The game window can be closed through the existing pygame.QUIT event in main.py. The shown handle_event() implementation does not provide a separate Exit key on the Game Over screen.

3. Assistant — Core Implementation / Code Changes

Code excerpts are source-based and shortened for readability; they are not presented as a complete source file.

```
# Source-based excerpt: set_difficulty() in game/game_engine.py

def set_difficulty(self, difficulty):
    self.difficulty = difficulty

    if difficulty == "Easy":
        self.spawn_interval = 70
        self.speed_scale = 0.8
        self.bomb_chance = 0.08

    elif difficulty == "Hard":
        self.spawn_interval = 40
        self.speed_scale = 1.2
        self.bomb_chance = 0.25

    else:
        # Medium
        self.spawn_interval = 55
        self.speed_scale = 1.0
        self.bomb_chance = 0.15


# Source-based excerpt: reset_game()
```

```
def reset_game(self, difficulty):
    # Clear the old game state
    self.fruits = []
    self.trail = []

    self._spawn_timer = 0
    self.lives = 3
    self.score = 0
    self.game_over = False

    # Apply selected difficulty
    self.set_difficulty(difficulty)


# Source-based excerpt: keyboard handling in handle_event()
elif event.type == pygame.KEYDOWN and self.game_over:

    if event.key == pygame.K_e:
        self.reset_game("Easy")

    elif event.key == pygame.K_m:
        self.reset_game("Medium")

    elif event.key == pygame.K_h:
        self.reset_game("Hard")
```

4. Assistant — Why It Works

Each difficulty changes the game's pace and risk level. Easy spawns less frequently, moves fruit more slowly, and uses a lower bomb probability. Hard increases the pace and bomb probability. Resetting the state prevents the previous round's score, lives, fruits, or swipe trail from carrying into the next round.

5. Testing Checklist

- ☐ Play until the Game Over screen appears.
- ☐ Press E in one round, M in another round, and H in another round.
- ☐ Confirm the selected difficulty's settings are applied.
- ☐ Verify that the score returns to zero and lives return to three.
- ☐ Confirm the old fruit list and swipe trail are cleared when replay starts.
- ☐ Close the game window and confirm the existing pygame.QUIT handling exits the game.

6. Task Outcome and Git Commit Message

Outcome: Players can choose Easy, Medium, or Hard after Game Over; the game state is reset before the next round.

Recorded commit message: `Add replay and difficulty levels`

Task 4 — Sound Effects

File(s): game/game_engine.py

1. User — Task Request

Add sound effects for slicing fruit and hitting a bomb, without requiring separate external WAV files. Also provide a sound when the player loses all lives.

2. Assistant — Explanation

The game sets up two sound objects during initialization. _setup_sounds() initializes the Pygame mixer if it is not already active and uses _create_tone() to generate WAV audio data in memory. A higher-pitched tone is played for a sliced fruit, while a lower-pitched tone is played when a bomb is sliced. The lower-pitched sound is also reused when lives reach zero. If audio initialization fails with pygame.error, the sound objects are set to None so sound playback can be skipped safely. The implementation does not define a separate game_over_sound.

3. Assistant — Core Implementation / Code Changes

Code excerpts are source-based and shortened for readability; they are not presented as a complete source file.

```
# Source-based excerpt: sound setup in __init__()

self.slice_sound = None
self.bomb_sound = None
self._setup_sounds()



# Source-based excerpt: relevant part of _setup_sounds()

if not pygame.mixer.get_init():
    pygame.mixer.init()

self.slice_sound = self._create_tone(
    frequency=900,
    duration=0.10,
    volume=0.30
)

self.bomb_sound = self._create_tone(
    frequency=180,
    duration=0.35,
    volume=0.40
)



# Source-based excerpt: relevant part of _slice()

if fruit.kind == "bomb":
    if self.bomb_sound:
        self.bomb_sound.play()

    self.game_over = True
```

```
else:
    if self.slice_sound:
        self.slice_sound.play()

    self.score += 1

# The implementation also plays self.bomb_sound when lives reach zero
# in update(). _create_tone() generates WAV data in memory and returns
# a Pygame Sound object. _setup_sounds() handles pygame.error by
# setting sound objects to None.
```

4. Assistant — Why It Works

The two tones provide immediate audio feedback for different game events. Generating the audio in memory avoids a dependency on separate sound files. Checking whether each sound object exists before calling play() allows the game logic to continue when audio setup is unavailable. The bomb sound is reused for the lives-zero event rather than introducing a separate Game Over sound.

5. Testing Checklist

- ☐ Run the game and slice a fruit; check for the higher-pitched sound.
- ☐ Slice a bomb; check for the lower-pitched sound and Game Over behavior.
- ☐ In a separate round, lose all lives; check that the lower-pitched sound plays.
- ☐ Check that the game can continue without sound playback if sound initialization fails.

6. Task Outcome and Git Commit Message

Outcome: Fruit slicing, bomb hits, and losing all lives receive audio feedback using generated sound objects.

Recorded commit message: `Add sound effects`

Combined Change Summary

- **Task 1 — Fast-swipe collision detection:** Samples intermediate positions along the mouse swipe segment and checks them against fruit hitboxes.
- **Task 2 — Game Over screen:** Adds a translucent overlay, GAME OVER title, final score, difficulty options, and replay instructions.
- **Task 3 — Replay and difficulty levels:** Adds Easy, Medium, and Hard settings, keyboard selection, and a reset method for a fresh round.
- **Task 4 — Sound effects:** Generates tones in memory and plays them for fruit slicing, bomb hits, and the lives-zero event.